

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA02

COURSE NAME: C Programming

COURSE OUTCOME COVERED

CO1: Construct structured C programs using constants, variables, data types, and preprocessor directives to address basic computational tasks. (BL2,BL3)

Assessment Tool Weightage: 20%

QUESTIONS:

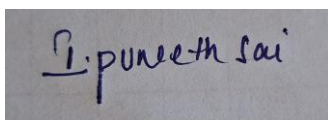
Total Marks:100

Code of Conduct:

I I PUNEETH SAI(192312267) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A rectangular box containing a handwritten signature in blue ink. The signature appears to be 'I. Puneeth Sai'.

1. Write a C program to:

- Display intermediate values using a macro DEBUG

Perform:

- Run with DEBUG = 1
- Run with DEBUG = 0

Record:

- Output differences

Conclude:

- How macros control program behavior
- Run with DEBUG = 1

The screenshot shows the CodeChef Online C Compiler interface. The code editor on the left contains the following C program:

```
1 #include <stdio.h>
2
3
4 #define DEBUG 1
5
6 int main()
7 {
8     int a = 10, b = 20, sum;
9     sum = a + b;
10
11     #if DEBUG
12         printf("DEBUG MODE\n");
13         printf("Value of a = %d\n", a);
14         printf("Value of b = %d\n", b);
15         printf("Intermediate Sum = %d\n", sum);
16     #endif
17
18     printf("Final Sum = %d\n", sum);
19     return 0;
20 }
21
22
23
```

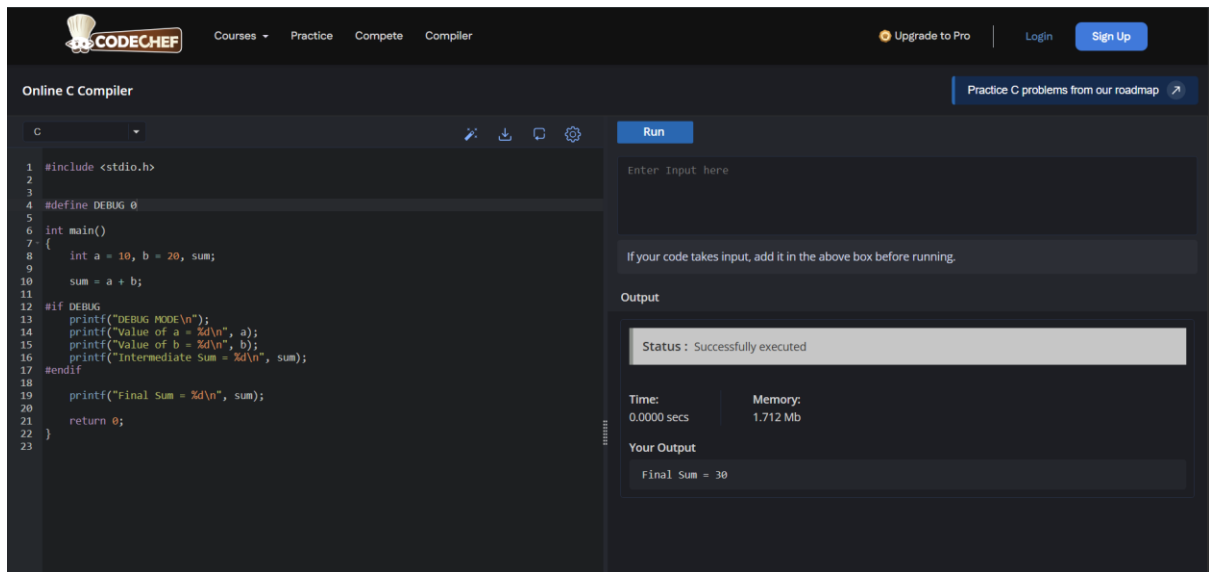
The right panel shows the execution results. The 'Run' button is highlighted. Below it, the 'Enter Input here' box is empty. A message states: 'If your code takes input, add it in the above box before running.' The 'Output' section shows the following output:

```
Status : Successfully executed

Time: 0.0000 secs    Memory: 1.652 Mb

Your Output
DEBUG MODE
Value of a = 10
Value of b = 20
Intermediate Sum = 30
Final Sum = 30
```

- Run with DEBUG = 0



Q2. Data Type Experiment

Write a program to calculate average of 3 numbers using:

- int
- float

Perform:

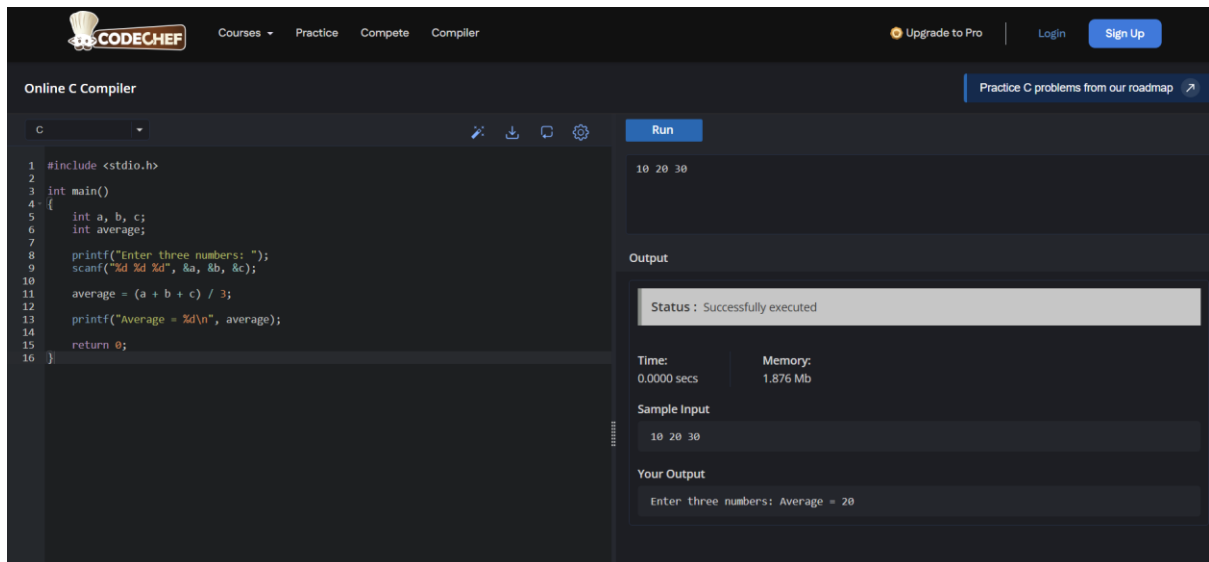
- Run both versions

Observe:

- Difference in output

Conclude:

- Effect of data types on accuracy
- int



CODECHEF Courses Practice Compete Compiler Upgrade to Pro Login Sign Up

Online C Compiler Practice C problems from our roadmap

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int a, b, c;
6     int average;
7
8     printf("Enter three numbers: ");
9     scanf("%d %d %d", &a, &b, &c);
10
11     average = (a + b + c) / 3;
12     printf("Average = %d\n", average);
13
14     return 0;
15 }
```

Run

10 20 30

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.876 Mb

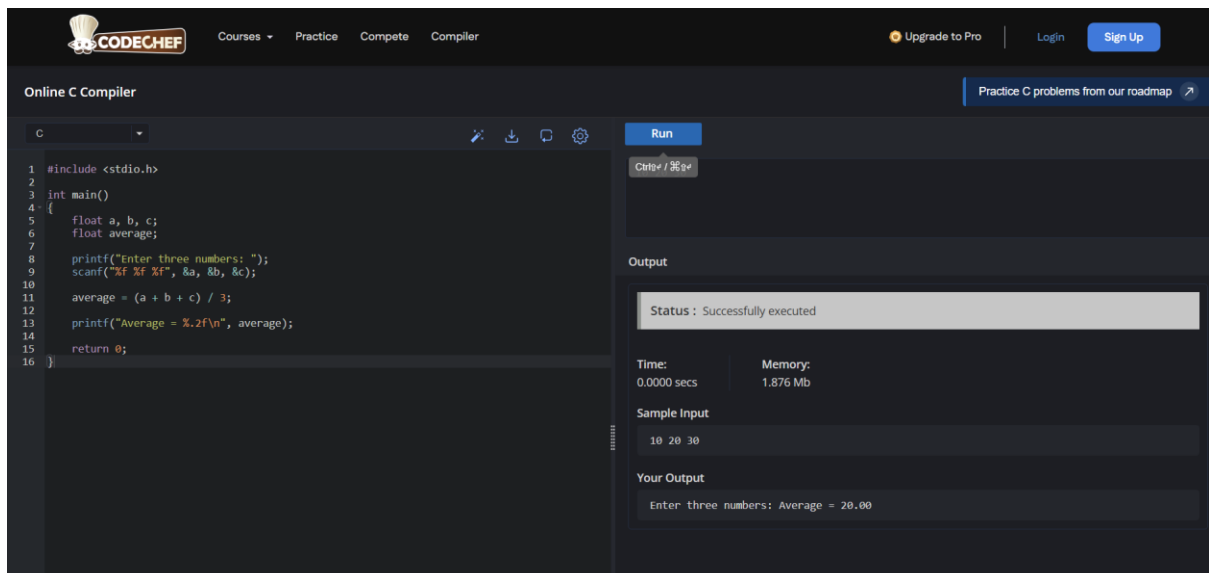
Sample Input

10 20 30

Your Output

Enter three numbers: Average = 20

- float



CODECHEF Courses Practice Compete Compiler Upgrade to Pro Login Sign Up

Online C Compiler Practice C problems from our roadmap

```
1 #include <stdio.h>
2
3 int main()
4 {
5     float a, b, c;
6     float average;
7
8     printf("Enter three numbers: ");
9     scanf("%f %f %f", &a, &b, &c);
10
11     average = (a + b + c) / 3;
12     printf("Average = %.2f\n", average);
13
14     return 0;
15 }
```

Run

Ctrl+R / ⌘+R

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.876 Mb

Sample Input

10 20 30

Your Output

Enter three numbers: Average = 20.00

Q3. Constant vs Variable Experiment

Write two versions of a program to compute the circumference of a circle:

- One using #define for constants
- One using variables

Perform:

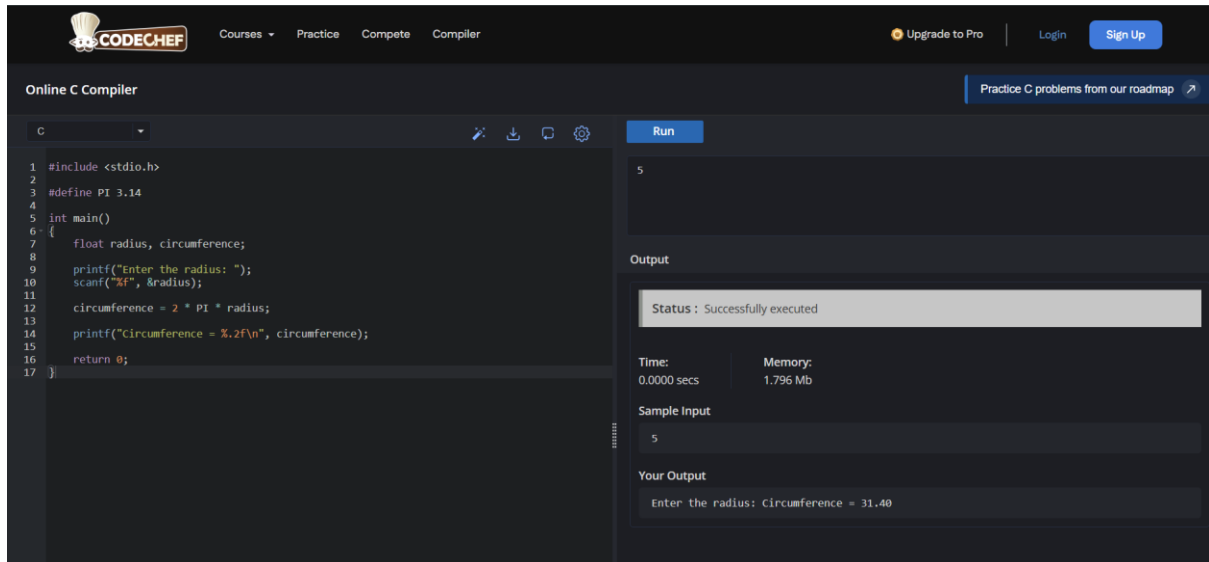
- Change values and run

Observe:

- Which is easier to modify

Conclude:

- Advantage of macros
- One using #define for constants



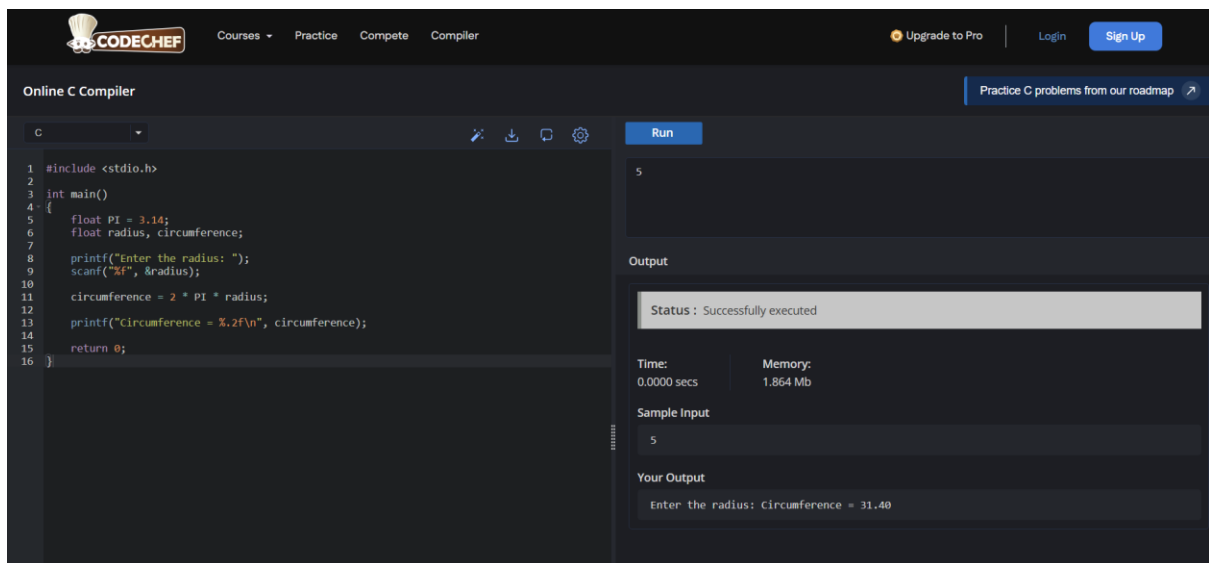
The screenshot shows the CodeChef Online C Compiler interface. The code in the editor is as follows:

```
1 #include <stdio.h>
2
3 #define PI 3.14
4
5 int main()
6 {
7     float radius, circumference;
8
9     printf("Enter the radius: ");
10    scanf("%f", &radius);
11
12    circumference = 2 * PI * radius;
13
14    printf("Circumference = %.2f\n", circumference);
15
16    return 0;
17 }
```

The right-hand panel shows the execution results:

- Run** button is active.
- Input:** 5
- Status:** Successfully executed
- Time:** 0.0000 secs
- Memory:** 1.796 Mb
- Sample Input:** 5
- Your Output:** Enter the radius: Circumference = 31.40

- One using variables



The screenshot shows the CodeChef Online C Compiler interface. The code in the editor is as follows:

```
1 #include <stdio.h>
2
3 int main()
4 {
5     float PI = 3.14;
6     float radius, circumference;
7
8     printf("Enter the radius: ");
9     scanf("%f", &radius);
10
11    circumference = 2 * PI * radius;
12
13    printf("Circumference = %.2f\n", circumference);
14
15    return 0;
16 }
```

The right-hand panel shows the execution results:

- Run** button is active.
- Input:** 5
- Status:** Successfully executed
- Time:** 0.0000 secs
- Memory:** 1.864 Mb
- Sample Input:** 5
- Your Output:** Enter the radius: Circumference = 31.40

Q4. Type Casting Experiment

Write a program to compute average:

- Without type casting
- With type casting

Observe:

- Output difference

Conclude:

- Why type casting is required
- Without type casting

The screenshot shows the CodeChef Online C Compiler interface. The code in the editor is as follows:

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int a, b, c;
6     float average;
7
8     printf("Enter three numbers: ");
9     scanf("%d %d %d", &a, &b, &c);
10
11     average = (a + b + c) / 3;
12
13     printf("Average = %.2f\n", average);
14
15     return 0;
16 }
```

The output of the program is 4.00. The status bar indicates "Status : Successfully executed". The execution time is 0.0000 secs and memory usage is 1.804 Mb.

- With type casting

The screenshot shows the CodeChef Online C Compiler interface. The code in the editor is as follows:

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int a, b, c;
6     float average;
7
8     printf("Enter three numbers: ");
9     scanf("%d %d %d", &a, &b, &c);
10
11     average = (float)(a + b + c) / 3;
12
13     printf("Average = %.2f\n", average);
14
15     return 0;
16 }
```

The output of the program is 4.33. The status bar indicates "Status : Successfully executed". The execution time is 0.0000 secs and memory usage is 1.808 Mb.

Q5. Input Validation Experiment

Write a program to:

- Accept input within a defined range using constants

Perform:

- Test with valid and invalid inputs

Observe:

- Program response

Conclude:

- Importance of validation

The screenshot shows the CodeChef Online C Compiler interface. The code editor on the left contains the following C program:

```

1 #include <stdio.h>
2
3 #define MIN 1
4 #define MAX 100
5
6 int main()
7 {
8     int num;
9
10    printf("Enter a number (%d to %d): ", MIN, MAX);
11    scanf("%d", &num);
12
13    if (num >= MIN && num <= MAX)
14    {
15        printf("Valid Input\n");
16        printf("You entered: %d\n", num);
17    }
18    else
19    {
20        printf("Invalid Input! Please enter a value between %d and %d.\n", MIN, MAX);
21    }
22
23    return 0;
24 }

```

The right side of the interface shows the execution results. The input field contains the number 105. The output field shows the program's response: "Valid Input\n" followed by "You entered: 105\n". The status bar indicates "Status: Successfully executed". The execution time is 0.0000 secs and the memory usage is 1.744 Mb. The sample input field also shows 105. The "Your Output" field shows the program's output: "Enter a number (1 to 100): Invalid Input! Please enter a value between 1 and 100."

Q6. Macro-Based Array Size

Write a program where array size is defined using macro

Perform:

- Change size and re-run

Observe:

- Output behavior

Conclude:

- Benefit of compile-time configuration

Size 5:

The screenshot shows the CodeChef Online C Compiler interface. The code in the editor is as follows:

```
1 #include <stdio.h>
2
3 #define SIZE 5
4
5 int main()
6 {
7     int arr[SIZE];
8     int i;
9
10    printf("Enter %d elements:\n", SIZE);
11    for(i = 0; i < SIZE; i++)
12    {
13        scanf("%d", &arr[i]);
14    }
15
16    printf("\nArray Elements are:\n");
17    for(i = 0; i < SIZE; i++)
18    {
19        printf("%d ", arr[i]);
20    }
21    return 0;
22 }
```

The output section shows the status "Successfully executed", execution time of 0.0000 secs, and memory usage of 1.808 Mb. The sample input is "10 20 30 40 50". The "Your Output" section shows the prompt "Enter 5 elements:" followed by the printed array elements "10 20 30 40 50".

Size 3:

The screenshot shows the CodeChef Online C Compiler interface with the code modified to read 3 elements:

```
1 #include <stdio.h>
2
3 #define SIZE 3
4
5 int main()
6 {
7     int arr[SIZE];
8     int i;
9
10    printf("Enter %d elements:\n", SIZE);
11    for(i = 0; i < SIZE; i++)
12    {
13        scanf("%d", &arr[i]);
14    }
15
16    printf("\nArray Elements are:\n");
17    for(i = 0; i < SIZE; i++)
18    {
19        printf("%d ", arr[i]);
20    }
21    return 0;
22 }
```

The output section shows the status "Successfully executed", execution time of 0.0000 secs, and memory usage of 1.876 Mb. The sample input is "10 20 30 40 50". The "Your Output" section shows the prompt "Enter 3 elements:" followed by the printed array elements "10 20 30".

Q7. Feature Enable/Disable Experiment

Write a program to read amount to be paid with

- Optional discount (if the amount is greater than 1000, give 5% discount) using macro

Perform:

- Enable and disable feature

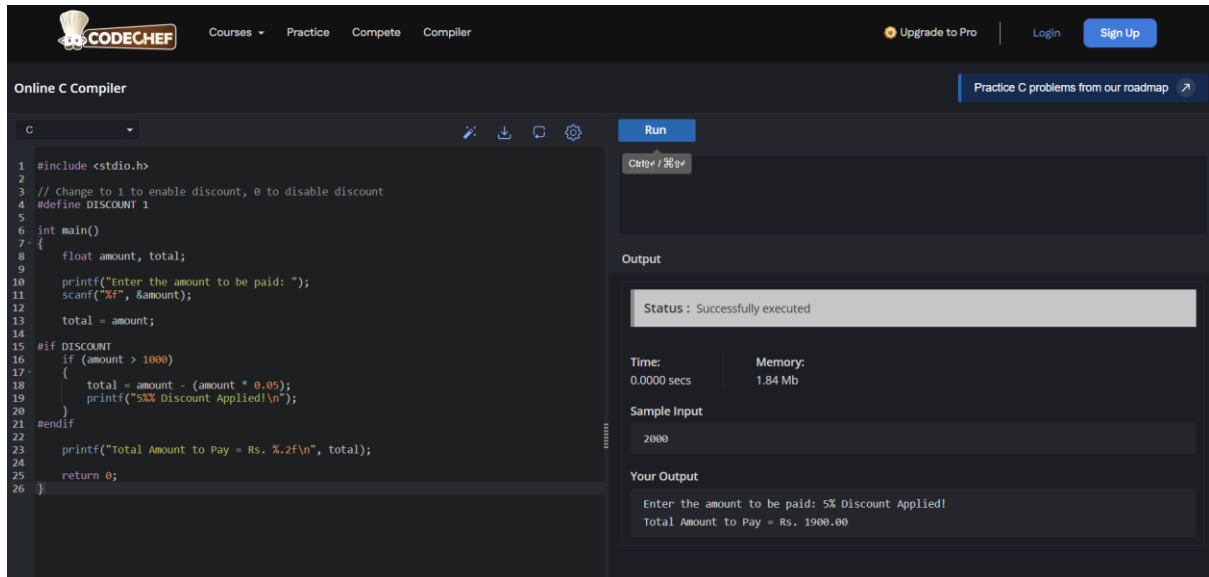
Observe:

- Change in total value

Conclude:

- Flexibility of preprocessor directives

Discount 1



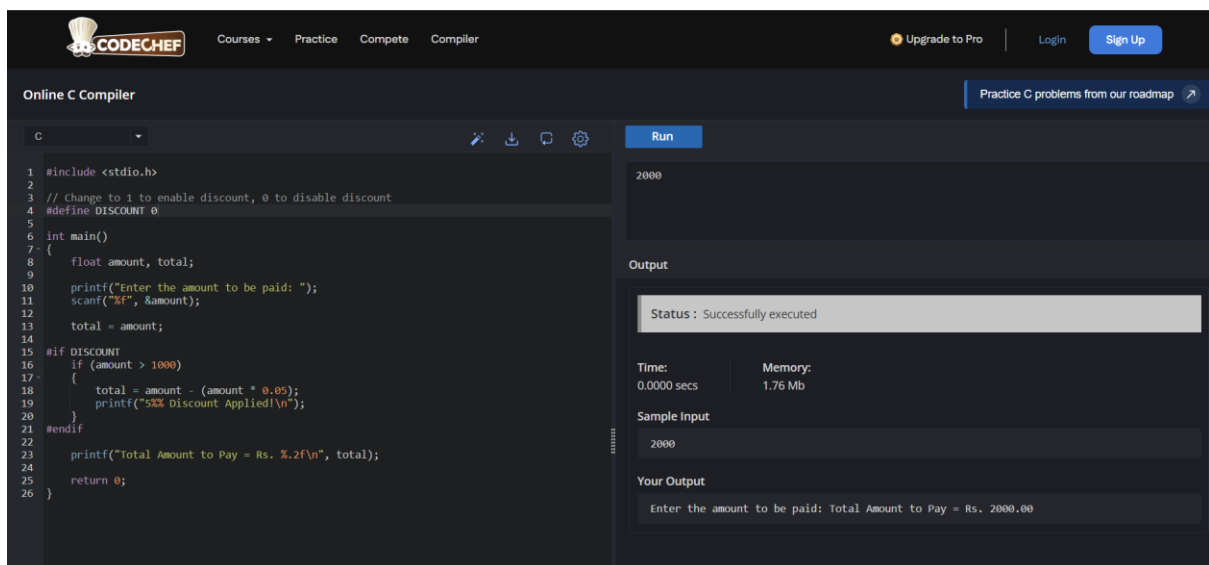
The screenshot shows the Online C Compiler interface. The code is as follows:

```
1 #include <stdio.h>
2
3 // Change to 1 to enable discount, 0 to disable discount
4 #define DISCOUNT 1
5
6 int main()
7 {
8     float amount, total;
9
10    printf("Enter the amount to be paid: ");
11    scanf("%f", &amount);
12
13    total = amount;
14
15    #if DISCOUNT
16    if (amount > 1000)
17    {
18        total = amount - (amount * 0.05);
19        printf("5% Discount Applied!\n");
20    }
21    #endif
22
23    printf("Total Amount to Pay = Rs. %.2f\n", total);
24
25    return 0;
26 }
```

The output shows the program was successfully executed. The sample input was 2000, and the output was:

```
Enter the amount to be paid: 5% Discount Applied!
Total Amount to Pay = Rs. 1900.00
```

Discount 0



The screenshot shows the Online C Compiler interface with the same code as above, but with the preprocessor directive changed to #define DISCOUNT 0. The output shows the program was successfully executed. The sample input was 2000, and the output was:

```
Enter the amount to be paid: Total Amount to Pay = Rs. 2000.00
```

Q8. Precision Experiment

Write a program to calculate area of circle

Perform:

- Use float and double

Observe:

- Precision difference

Conclude:

- Best data type choice

Float

Q8. Precision Experiment

Write a program to calculate area of circle

Perform:

- Use float and double

Observe:

- Precision difference

Conclude:

- Best data type choice

Float :

The screenshot shows the CodeChef Online C Compiler interface. The code in the editor is as follows:

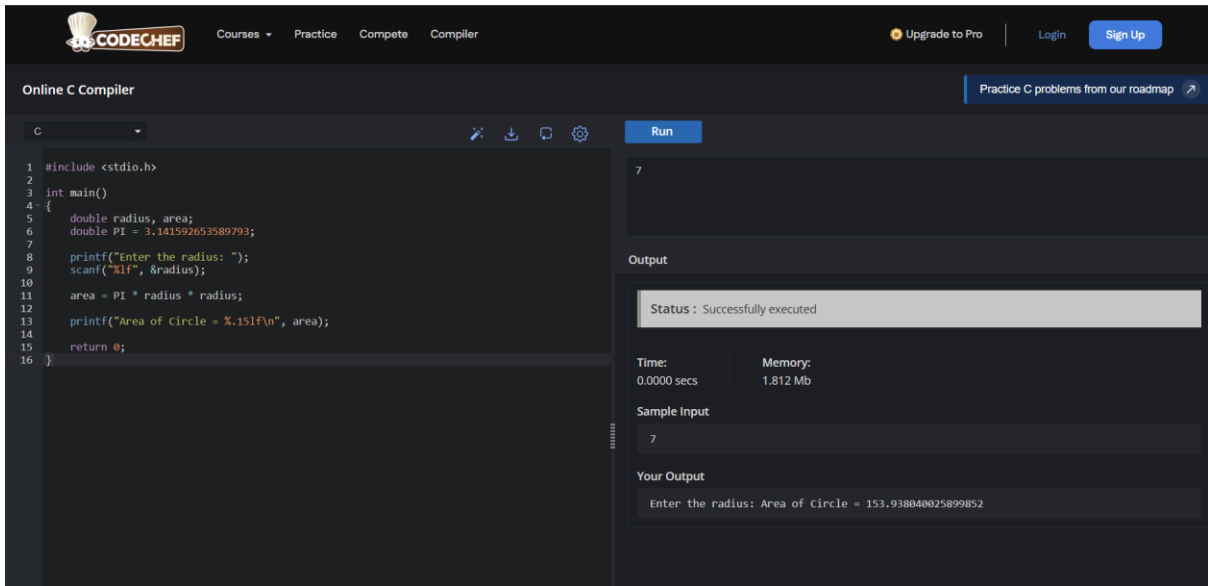
```

1 #include <stdio.h>
2
3 int main()
4 {
5     float radius, area;
6     float PI = 3.14159265f;
7
8     printf("Enter the radius: ");
9     scanf("%f", &radius);
10
11     area = PI * radius * radius;
12
13     printf("Area of Circle = %.6f\n", area);
14
15     return 0;
16 }

```

The input provided is 7. The output shows the status "Successfully executed" and performance metrics: Time: 0.0000 secs, Memory: 1.8 Mb. The "Your Output" section displays the prompt "Enter the radius: Area of Circle = 153.938049", indicating a loss of precision due to the use of float.

Double:



The screenshot shows the CodeChef Online C Compiler interface. The code editor on the left contains a C program that calculates the area of a circle. The program uses a double variable for PI and a double variable for the radius. The output on the right shows the status as 'Successfully executed', the time as 0.0000 secs, and the memory as 1.812 Mb. The sample input is 7, and the output is 'Enter the radius: Area of Circle = 153.938848025899852'.

```
1 #include <stdio.h>
2
3 int main()
4 {
5     double radius, area;
6     double PI = 3.141592653589793;
7
8     printf("Enter the radius: ");
9     scanf("%lf", &radius);
10
11     area = PI * radius * radius;
12
13     printf("Area of Circle = %.15lf\n", area);
14
15     return 0;
16 }
```

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.812 Mb

Sample Input

7

Your Output

Enter the radius: Area of Circle = 153.938848025899852

Q9. Overflow Experiment

Write a program to:

- Store large values in different data types

Perform:

- Test with large inputs

Observe:

- Overflow behavior

Conclude:

- Limitations of data types

```
1 #include <stdio.h>
2 #include <limits.h>
3
4 int main()
5 {
6     int intValue;
7     short shortValue;
8     long longValue;
9
10    // Assigning large values
11    intValue = INT_MAX;
12    shortValue = SHRT_MAX;
13    longValue = 9223372036854775807LL;
14
15    printf("Maximum int value : %d\n", intValue);
16    printf("Maximum short value : %d\n", shortValue);
17    printf("Maximum long value : %lld\n", longValue);
18
19    // Overflow operation
20    printf("\nAfter Overflow:\n");
21
22    intValue = intValue + 1;
23    shortValue = shortValue + 1;
24    longValue = longValue + 1;
25
26    printf("Integer value after overflow : %d\n", intValue);
27    printf("Short value after overflow : %d\n", shortValue);
28    printf("Long value after overflow : %lld\n", longValue);
29
30    return 0;
31 }
```

1213644136
12558932486
16843696781698

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.644 Mb

Sample Input

1213644136
12558932486
16843696781698

Your Output

Maximum int value : 2147483647
Maximum short value : 32767
Maximum long value : 9223372036854775807

After Overflow:
Integer value after overflow : -2147483648
Short value after overflow : -32768
Long value after overflow : -9223372036854775808

Q10. Multi-Case Testing

Write a program to:

- Convert temperature (Kelvin to Celcius, Kelvin to Farenheit)

Perform:

- Test with 3 different inputs

Observe:

- Output correctness

Conclude:

- Program reliability

```

1 #include <stdio.h>
2
3 float kelvinToCelsius(float k)
4 {
5     return k - 273.15;
6 }
7
8 float kelvinToFahrenheit(float k)
9 {
10    return (k - 273.15) * 9 / 5 + 32;
11 }
12
13 int main()
14 {
15     float kelvinValues[3] = {273.15, 300, 373.15};
16
17     printf("Kelvin Temperature Conversion Testing\n");
18     printf("-----\n");
19
20     for(int i = 0; i < 3; i++)
21     {
22         float kelvin = kelvinValues[i];
23
24         printf("Kelvin: %.2f K\n", kelvin);
25         printf("Celsius: %.2f C\n", kelvinToCelsius(kelvin));
26         printf("Fahrenheit: %.2f F\n\n", kelvinToFahrenheit(kelvin));
27     }
28
29     return 0;
30 }

```

Sample Input

```

21K
231k
125k

```

Your Output

```

Kelvin Temperature Conversion Testing
-----
Kelvin: 273.15 K
Celsius: -0.00 C
Fahrenheit: 32.00 F

Kelvin: 300.00 K
Celsius: 26.85 C
Fahrenheit: 80.33 F

```

Q11. Replace Hardcoding

Write a program to swap two values. Hard code the values.

Modify:

- Replace with macros

Observe:

- Code readability

Conclude:

- Importance of symbolic constants

```

1 #include <stdio.h>
2
3 int main()
4 {
5     int a = 10;
6     int b = 20;
7     int temp;
8
9     printf("Before swapping:\na = %d\nb = %d\n", a, b);
10
11     temp = a;
12     a = b;
13     b = temp;
14
15     printf("After swapping:\na = %d\nb = %d\n", a, b);
16
17     return 0;
18 }

```

Sample Input

```

Enter Input here

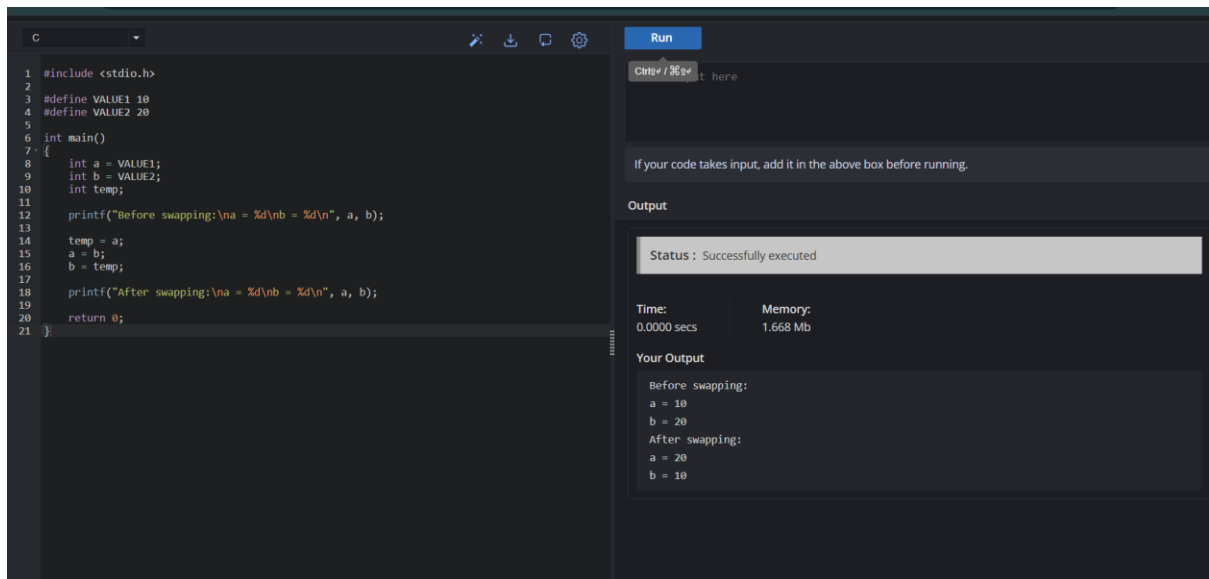
```

Your Output

```

Before swapping:
a = 10
b = 20
After swapping:
a = 20
b = 10

```



```
1 #include <stdio.h>
2
3 #define VALUE1 10
4 #define VALUE2 20
5
6 int main()
7 {
8     int a = VALUE1;
9     int b = VALUE2;
10    int temp;
11
12    printf("Before swapping:\na = %d\nb = %d\n", a, b);
13
14    temp = a;
15    a = b;
16    b = temp;
17
18    printf("After swapping:\na = %d\nb = %d\n", a, b);
19
20    return 0;
21 }
```

Run

Ctrl + Shift + R

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:	Memory:
0.0000 secs	1.668 Mb

Your Output

```
Before swapping:
a = 10
b = 20
After swapping:
a = 20
b = 10
```

Q12. Control Structure Experiment

Write a program to read three marks and calculate the grade based on the following constraints:

- if the student fails in any one subject his grade is displayed as 'F'.
- if the average is > 90, display 'S'
- if the average is > 80, display 'A'
- if the average is > 70, display 'B'

Perform:

- Test boundary values

Observe:

- Output correctness

Conclude:

- Role of conditions

The screenshot shows a C program in the CodeChef online compiler. The program prompts the user to enter marks for three subjects. It calculates the average and assigns a grade based on the average. The output shows the calculated grade 'F' for the input marks 10, 20, and 30.

```
1 #include <stdio.h>
2
3 int main()
4 {
5     float mark1, mark2, mark3, average;
6     char grade;
7
8     printf("Enter marks of three subjects: ");
9     scanf("%f %f %f", &mark1, &mark2, &mark3);
10
11     if(mark1 < 40 || mark2 < 40 || mark3 < 40)
12     {
13         grade = 'F';
14     }
15     else
16     {
17         average = (mark1 + mark2 + mark3) / 3;
18         if(average > 90)
19             grade = 'S';
20         else if(average > 80)
21             grade = 'A';
22         else if(average > 70)
23             grade = 'B';
24         else
25             grade = 'C';
26     }
27
28     printf("Grade = %c\n", grade);
29     return 0;
30 }
31
32 }
```

Input: 10 20 30 70
5

Output: Status : Successfully executed
Time: 0.0000 secs Memory: 1.804 Mb
Sample Input: 10 20 30 70
5
Your Output: Enter marks of three subjects: Grade = F

Q13. Step-by-Step Output Tracking

Write a program to calculate the electricity consumption.

Add:

- Print statements using macro

Observe:

- Intermediate results

Conclude:

- Debugging benefits

The screenshot shows a C program in the CodeChef online compiler. The program calculates the electricity bill based on units consumed and cost per unit. It includes debug statements to print intermediate values. The output shows the calculated bill of 750.00 for 150 units at a rate of 5.00.

```
1 #include <stdio.h>
2
3 #define DEBUG(x) printf("%s = %.2f\n", #x, x)
4
5 int main()
6 {
7     float units, rate, bill;
8
9     printf("Enter electricity units consumed: ");
10    scanf("%f", &units);
11
12    printf("Enter cost per unit: ");
13    scanf("%f", &rate);
14
15    DEBUG(units);
16    DEBUG(rate);
17
18    bill = units * rate;
19
20    DEBUG(bill);
21
22    printf("Total Electricity Bill = %.2f\n", bill);
23    return 0;
24 }
25 }
```

Input: 150
5

Output: Status : Successfully executed
Time: 0.0000 secs Memory: 1.872 Mb
Sample Input: 150
5
Your Output: Enter electricity units consumed: Enter cost per unit: units = 150.00
rate = 5.00
bill = 750.00
Total Electricity Bill = 750.00

Q14. Minimal Memory Design

Write a program that reads cost of an item, number of items bought and calculate the total amount to be paid:

Apply:

- smallest suitable data types

Perform:

- Compare with normal version

Observe:

- Efficiency

Conclude:

- Importance of data selection

The screenshot shows a web-based C compiler interface. The code editor on the left contains the following C program:

```
1 #include <stdio.h>
2
3 int main()
4 {
5     float cost, total;
6     unsigned short int quantity;
7
8     printf("Enter cost of one item: ");
9     scanf("%f", &cost);
10
11     printf("Enter number of items: ");
12     scanf("%hu", &quantity);
13
14     total = cost * quantity;
15
16     printf("Total amount = %.2f\n", total);
17
18     return 0;
19 }
```

On the right side, there is a 'Run' button. Below it, the 'Input' section shows the values 50 and 4. The 'Output' section displays 'Status: Successfully executed'. Below that, the 'Time' is 0.0000 secs and 'Memory' is 1.872 Mb. The 'Sample Input' section shows 50 and 4. The 'Your Output' section shows the result: 'Enter cost of one item: Enter number of items: Total amount = 200.00'.

```
1 #include <stdio.h>
2
3 int main()
4 {
5     double cost, total;
6     int quantity;
7
8     printf("Enter cost of one item: ");
9     scanf("%lf", &cost);
10
11    printf("Enter number of items: ");
12    scanf("%d", &quantity);
13
14    total = cost * quantity;
15
16    printf("Total amount = %.2lf\n", total);
17
18    return 0;
19 }
```

Run

50
4

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.856 Mb

Sample Input

50
4

Your Output

Enter cost of one item: Enter number of items: Total amount = 200.00

Q15. Program Modification Task

Write a basic program to find the celsius using Farenheit

Modify:

- Add macros
- Change data types

Observe:

- Output changes

Conclude:

- Impact of design choices

codechef.com/c-online-compiler

Ask Google

C

```
1 #include <stdio.h>
2
3 int main()
4 {
5     float fahrenheit, celsius;
6
7     printf("Enter temperature in Fahrenheit: ");
8     scanf("%f", &fahrenheit);
9
10    celsius = (fahrenheit - 32) * 5 / 9;
11
12    printf("Temperature in Celsius = %.2f\n", celsius);
13
14    return 0;
15 }
```

Run

98.3

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.796 Mb

Sample Input

98.3

Your Output

Enter temperature in Fahrenheit: Temperature in Celsius = 36.83

C

Ctrl+ / 98.3

C

```
1 #include <stdio.h>
2
3 #define OFFSET 32
4 #define SCALE 5.0
5 #define DIVISOR 9.0
6
7 int main()
8 {
9     double fahrenheit, celsius;
10
11    printf("Enter temperature in Fahrenheit: ");
12    scanf("%lf", &fahrenheit);
13
14    celsius = (fahrenheit - OFFSET) * SCALE / DIVISOR;
15
16    printf("Temperature in Celsius = %.2lf\n", celsius);
17
18    return 0;
19 }
```

Run

Ctrl+ / 98.3

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.82 Mb

Sample Input

98.3

Your Output

Enter temperature in Fahrenheit: Temperature in Celsius = 36.83